

A Relation-Centred Graph Tower for Finite-Volume Simulation

with Exact Derivatives of Any Order

Architecture, mathematics, verification and an engineering assessment of UFVM

Komahan Boopathy

Technical report, 22 August 2026 — source at revision `db3e31f` with the working tree of this date

Abstract

UFVM is a Fortran 2023 library for finite-volume simulation on unstructured meshes in which every object is a composition of eight *prime* types—graph, token, relation, field, operation, transform, map and view—rather than a member of a growing taxonomy of solver classes. The mesh is not stored but *derived*: a file supplies member sets, one cell-to-vertex relation, vertex coordinates and boundary tags, and relation algebra produces faces, incidences and every geometric measurement. A sparse matrix is a weighted relation; a differential operator is a composition of three elementary affine maps between the vertex and edge sets; a time integrator is a traversal of a chain graph; and the adjoint of any compiled operator—in space or in time—is the transpose of the composition, so no reversed kernel exists anywhere in the source. Differentiability is a capability of every operation: a statement declares its exact partial actions once, and tangents, Newton Jacobians, adjoints and forward directional derivatives of arbitrary order are *derived* from them through the chain rule over integer partitions and the implicit function theorem. The same two implicit-function solves serve a time step and a steady spatial residual. We present the mathematical architecture, the design laws that make code disappear, and measurements: the three marching rules reach their nominal orders (1, 1, 2); the discrete adjoint of backward Euler matches its closed form to machine precision and finite differences to 4×10^{-11} ; second-order directional derivatives in a rate parameter match closed form to 6×10^{-17} ; the compiled diffusion operator is second order on meshes of 10^2 to 80^2 cells; and the 159 property checks of the project’s verification suites—removed from the tree in August 2026 and restored from revision `ddcf592` by a script shipped with this report—pass against the current source. The assessment also records what is not yet right: operator assembly scales quadratically with cell count, the unpreconditioned Krylov solve scales as $n^{1.7}$, the in-repository test tree was removed, and the distributed runtime is designed but not implemented.

Contents

1	Introduction	2
2	Motivation	3
3	Related work and positioning	3
4	The mathematical architecture	4
4.1	Eight primes and the composition arithmetic	4
4.2	The kernel graph and identity	5
4.3	Sets, maps and views	6
4.4	Relations	6
4.5	The partition relation	7

4.6	Fields, reductions and the parallel law	7
4.7	Operations and differentiability	8
4.8	The affine sparse map	8
4.9	The scheme and the chain graph	9
4.10	The implicit function theorem, once	10
4.11	The marcher as a traversal	10
4.12	Minimizers and multigrid	10
4.13	The mesh, derived	11
4.14	Physics as coefficients	11
4.15	Reversible change	11
5	What the system can do	12
6	Results	12
6.1	Verification suites	12
6.2	Time integration: orders of accuracy	13
6.3	Derivatives: exactness of the adjoint and of directional derivatives	13
6.4	Space: a Poisson problem through the constitution	14
6.5	Cost and scaling	15
6.6	The source as an artefact	15
6.7	The two unification slices	15
7	Discussion: what is novel, and what is different	16
8	Engineering assessment	16
9	Conclusions	17
10	Future work, from an engineering perspective	17
A	Interface changes applied by the oracle script	18

1 Introduction

Scientific simulation codes tend to grow by accretion. A finite-volume solver starts with a mesh class, adds a field class, then a boundary-condition hierarchy, a linear-solver hierarchy, a time-integrator hierarchy, and, much later, sensitivity analysis: tangent and adjoint versions of each of those, produced by hand, by source transformation, or by operator-overloaded tapes. Each addition is a new kind of object with its own invariants, and the coupling between kinds is what makes such codes expensive to extend and hard to verify.

UFVM takes the opposite route. It fixes a small set of irreducible mathematical objects and requires everything else to be a composition, a view, or a concretion of them. The consequence that matters most for engineering is that mathematical laws become *code-elimination mechanisms*: when $(F \circ G)^* = G^* \circ F^*$ is an identity of the representation, the reverse kernel is not written; when a transpose is a view of a relation, the second CSR is not stored; when Newton's Jacobian is the tangent of the statement, no Jacobian is hand-coded; when the dependency graph of a matrix is derivable from its entries, it is not kept. The source as it stands is 63 modules and 21 450 lines, of which 27% is prose stating the laws each module keeps.

This report is the first complete description of the system as it exists. It aims to answer four questions a prospective user or maintainer would ask: what the library can do (Section 5), on what mathematics it rests (Section 4), what has been verified and measured (Section 6), and

what is novel about it compared with established frameworks (Sections 3 and 7). It closes with an engineering assessment and a prioritized list of future work (Sections 8–10).

Contributions.

1. A *prime decomposition* of finite-volume simulation into eight types with a stated composition arithmetic, a self-similar kernel graph, and unforgeable identity (Section 4.1).
2. A *derived mesh*: faces, incidences and all measurements computed by relation algebra from one relation and one coordinate field (Section 4.13).
3. *Differentiability as a capability of every operation*: exact partial actions declared once, every derivative object derived—tangent, Jacobian, adjoint, higher-order directional derivatives—with no derivative tensor stored and no automatic-differentiation tool (Section 4.7).
4. *One affine sparse map* for space and time, carrying composition and transpose once, so the adjoint of a compiled chain is never re-implemented (Section 4.8).
5. *The implicit function theorem as two routines* that serve a time step and a steady residual identically (Section 4.10).
6. Verification and measurement of orders of accuracy, derivative exactness and cost, with the scaling defects reported as found (Section 6).

2 Motivation

Three recurring problems of simulation software motivated the design.

Taxonomies do not compose. A class per physical notion—`Mesh`, `Field`, `BoundaryCondition`, `Solver`, `Integrator`—produces $O(k^2)$ interactions between k kinds, each a special case. The alternative pursued here is to ask which notions are *irreducible*. A mesh is a graph with measurements; a matrix is a graph with weights; a time axis is a graph; a boundary condition is a relation between faces and coefficients. If all of these are one kernel object read through different views, the interactions are theorems rather than cases.

Derivatives are retrofitted. Sensitivities are usually added after the primal code exists, by a tool that differentiates source text or by a tape that records operations at run time. Both produce a second program whose correctness must be established separately, and neither exploits the structure that the primal already has: that every discrete operator is a composition of a few elementary maps, that a linear map is its own tangent, that an implicit step is one local solve. UFVM makes the partial derivative of a statement part of the statement’s contract, and derives everything else.

Time and space are treated by different machinery. An integrator is written as a loop over steps, a spatial operator as a loop over faces, and their adjoints as two more loops. Both are traversals of a graph in which a local statement reads its neighbours; the adjoint of both is the converse traversal with transposed local blocks. When the representation says so, one implementation serves both.

The governing principle, stated in the repository’s constitution and enforced in its source, is: *prove structure once; delete the corresponding cases from the implementation.*

3 Related work and positioning

Finite-volume frameworks. OpenFOAM [1] and SU2 [2] are the reference open-source unstructured finite-volume codes. Both store the mesh explicitly (owner/neighbour or cell-face adjacency arrays) and express discretizations through operator objects that assemble matrices.

SU2 obtains discrete adjoints by operator-overloading algorithmic differentiation (CoDiPack [3]) of the whole primal iteration; OpenFOAM’s adjoint solvers are hand-derived continuous or discrete adjoints per physics. In UFVM the mesh is derived from one relation, matrices are weighted relations, and the adjoint of a compiled operator is its transpose.

Finite-element frameworks with symbolic forms. FEniCS [4], Firedrake [5] and MFEM [6] express variational forms symbolically (UFL) or through operator classes and generate kernels. dolfin-adjoint/pyadjoint [7, 8] derive discrete adjoints by recording the sequence of solves on a tape and reversing it; higher-order derivatives are obtained by taping the tangent. UFVM shares the idea that the derivative should follow from the structure of the forward computation, but realizes it without a tape: each operation declares its exact partial actions, and the chain rule to any order is assembled from integer partitions at the call.

Time integration with sensitivities. SUNDIALS CVODES/IDAS [9] and PETSc TS with TSAdjoint [10] provide forward and adjoint sensitivities of multistep and Runge–Kutta integrators, with hand-derived adjoint formulas per method family. In UFVM the θ -scheme and BDF family are one operation whose partials are derived from the action’s, and the adjoint march is the converse traversal with the transposed compiled block; no integrator has an adjoint of its own.

Algorithmic differentiation. Source transformation (Tapenade [11]), operator overloading (ADOL-C [12], CoDiPack), compiler-level differentiation (Enzyme [13]) and array-language transformation (JAX [14]) differentiate programs. UFVM differentiates *statements*: the unit is a mathematical operation with a declared argument space, and the derivative machinery is structural (composition, transpose, partitions) rather than program-level. The approach is closer in spirit to Griewank and Walther’s treatment of higher-order derivatives through Taylor arithmetic [15] and to Faà di Bruno’s formula, which the chain-rule module implements directly.

Discrete exterior calculus. The three elementary steps—average, difference, incidence—are the 0- and 1-cochain operators of discrete exterior calculus [16, 17, 18]. UFVM does not adopt the full DEC vocabulary; it keeps the operators as affine maps between two finite sets so that the boundary value enters the constant and the adjoint is the transpose of the composition.

Relational and graph-based formulations. Treating the mesh as relations between sets and deriving incidence by relational algebra is familiar from the database view of meshes and from the topological data structures literature [19]. UFVM takes it further: relations are first-class with identity, the partition of a graph is one relation read in two directions, and a weighted relation is the matrix.

Table 1 summarizes the positioning on the axes that distinguish the design. The table records mechanisms, not quality; each of the systems named is far more complete in physics and scale than UFVM today.

4 The mathematical architecture

4.1 Eight primes and the composition arithmetic

Definition 1 (prime). *A type is prime when it cannot be expressed as a composition of the others. Everything that is not prime must say which primes it composes.*

The eight primes and the arithmetic that generates the rest are shown in Figure 1. The statement that every higher type is a composition is enforced by two mechanical checks on the source: the build order of the 63 modules is a linear extension of their dependency graph (a file

Axis	OpenFOAM, SU2	FEniCS, Firedrake	SUNDIALS, PETSc TS	UFVM
Mesh	stored adjacency arrays	stored topology	n/a	derived from one relation and one field
Matrix	sparse storage class	sparse storage class	PETSc Mat	weighted relation; pattern derived
Derivative mechanism	hand adjoints; AD by overloading (SU2)	symbolic forms; tape (pyadjoint)	hand-derived method	per declared partial actions; structural composition
Adjoint of an operator chain	differentiated program	reversed tape	method-specific formulas	for- transpose of the composed map
Higher-order derivatives	tape of tape	tape of tape	second order (TSAdjoint)	any order, Faà di Bruno over partitions
Time vs. space	separate machinery	separate machinery	time only	one traversal, one implicit-function solve
Language	C++	Python/C++	C	Fortran 2023

Table 1: Positioning by mechanism. The comparison is about how each system represents and differentiates, not about scope or maturity.

may import only modules built before it), and the naming law assigns every module to a prime namespace.

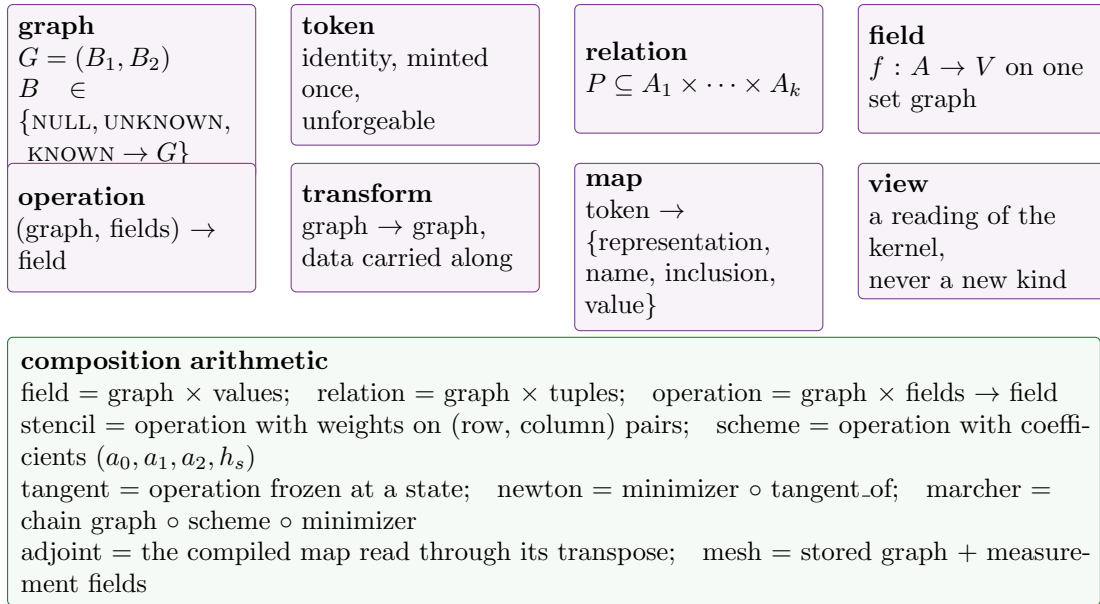


Figure 1: The eight primes and the arithmetic by which every other type is formed. The kernel graph is self-similar: sets, tuples, relations, domains and chains are this one object read differently, and there is exactly one type named **graph**.

4.2 The kernel graph and identity

The kernel is deliberately minimal (Figure 2). A graph is a pair of branches; a branch is NULL, UNKNOWN, or a reference to another graph. The three states are values, not subtypes; the status and the reference are private, so the invariant

$$\text{status} = \text{KNOWN} \iff \text{the reference is associated}$$

can only be established by the three constructors and cannot be broken by a caller. UNKNOWN is distinct from NULL: a halo before exchange is not absent, and an empty set is not unknown.

Identity is a service beneath the tower. A token is an (image, serial) pair with private components, so the only ways a token exists are minting and copying: a copy *is* the identity, and two coarray images can never mint the same stamp. Every map in the system is keyed on identity tokens copied at attachment, never on storage position, so containers may grow, reorder or compact without redirecting an attachment. This separates two questions that simulation codes habitually conflate:

$$\text{identity (which set?)} \neq \text{extension (which members?)}.$$

Two independently declared sets over $\{1, \dots, 8\}$ are equivalent and are not the same set; a transposed relation signs its own token and is never `same_as` its base even though it has the same tuples.

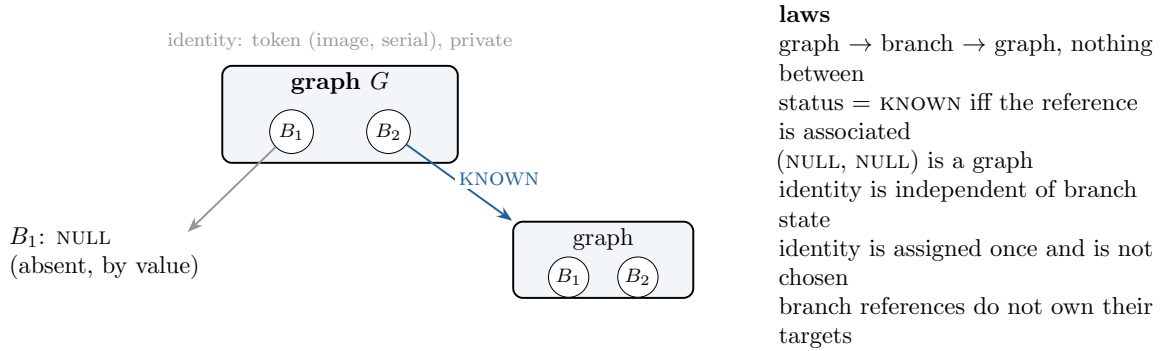


Figure 2: The kernel graph. Sets, sequences, relational graphs and epistemic (data, operator) pairs are *views* of this object; the view that reads a graph as a set stores the extension in a representation outside the graph, so a set of N members is $O(1)$ semantic objects, not $O(N)$.

4.3 Sets, maps and views

A set is a graph read through a *representation* held in a map: `num_members`, `member(k)` and `local_index(v)` bound by the enumeration laws

$$\text{member}(\text{local_index}(v)) = v, \quad \text{local_index}(\text{member}(k)) = k,$$

from which membership and the member list are theorems answered once for every concretion. A counted representation answers `local_index` in $O(1)$; a listed one scans. Three further maps record what a set is called, which ambient set a subset is declared into (the inclusion is the identity on values, so only the *association* is stored), and which value field is attached to a graph with what trust status. All are keyed on identity and stored outside the graph.

4.4 Relations

A relation is a named finite-arity subset $P \subseteq A_1 \times \dots \times A_k$ with identity, an ordered signature of set graphs, and a membership law; it is first-class and needs no graph to exist. Arity two earns its own contract because it has its own questions: `source`, `target`, `image(a)`, `preimage(b)`. The CSR concretion stores both directions once at construction, so each fibre is one row slice and construction is linear in members plus tuples; every query speaks member values, never storage rows, and the cost is stated parametrically, $T_{\text{image}}(a) = T_{\text{local_index}}(a) + O(\deg a)$.

Law 1 (transpose is a view). *transpose_of(P) is an $O(1)$ view that borrows its base: image and preimage swapped, the signature read in reverse. The involution $(P^T)^T = P$ is a statement about extension, not identity.*

One grouping kernel—a counting sort that fibres a tuple list over one slot—builds the CSR, the incidence lists, the padded transpose and the combination of duplicate matrix entries; no other count/prefix/scatter exists in the source. The relation algebra module holds exactly the three primitives its first caller earned (restriction, projection, binary composition); graph algorithms (sources, sinks, reachability, topological order with loud refusal on a cycle) act on a relation from outside, as free procedures, never as methods.

4.5 The partition relation

A cut and an assembly are not two records but one relation read in two directions, $r \subseteq S_{\text{part}} \times S_{\text{whole}}$, one per carrier (vertices, edges). The partitioner writes it; the assembler is handed it and never invents one. The law the pair satisfies,

$$\text{assemble}(\text{partition}(G)) = G, \quad \text{assemble}(\text{partition}(G, D)) = (G, D),$$

is exact in both directions; coarsen/refine satisfy the one-sided law $\text{coarsen}(\text{refine}(G)) = G$. A part owns the cells it answers for and borrows the neighbours it must see; only owned values are collected, because collecting a borrowed copy would count a conserved quantity twice—exactly at a partition boundary, where the error is hardest to locate (Figure 3).

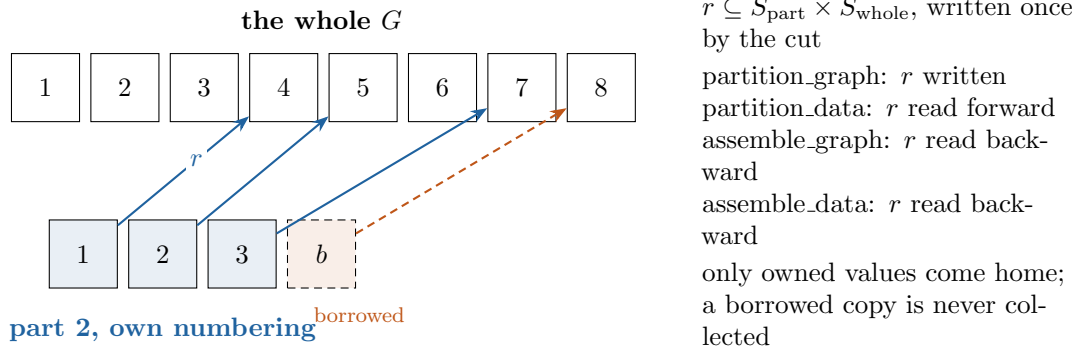


Figure 3: The partition relation. One value drives all four motions; the round-trip law is exact. The suite verifies, for 1–3 parts, that every assembled edge is an edge of G and every edge of G is assembled by some part, that $\sum_k A_k(P_k(D)) = D$ on vertex and edge data, that every cell is owned exactly once, and that a part handed the wrong relation is *refused by identity*, the two relations being indistinguishable by size.

4.6 Fields, reductions and the parallel law

A field is a function over one finite domain named by a set graph; it keeps the domain’s identity and a frozen count, nothing else, so N fields on one domain do not hold N copies of its extension. A functional is the field at domain size one; it carries real, complex (so a complex-step derivative survives a reduction) and logical values.

A reduction turns a field into a number in four steps—accumulate, combine, finalize, with a measure—because the graph may be in pieces. The average of part-averages is wrong; the sum and the count must travel together and divide once at the end. Minimum and maximum are safe to finish early, average and norm are not, so every rule uses the four steps. With a measure a bare sum becomes an integral, and the measure position is also the inner product’s second field, which is how the Krylov solvers obtain $\langle q, v \rangle$ from the reduction. This is the parallel law, designed in before any parallel code exists.

4.7 Operations and differentiability

An operation is the verb within a graph: $(\text{graph}, \text{fields}) \rightarrow \text{field}$, with three symbols—**name**, **domain** (which set the result lives on and how many entries) and **apply**. A concrete operation is handed the fields it reads at construction; **apply** fetches nothing by name.

The operation also owns an *argument space*. $F(x_1, \dots, x_m)$ reads inputs by position; **declare_arguments**(m) fixes the readable positions; an argument is an opaque ordinal in one operation’s space, and two arguments match only when they name the same position of the same space—an argument of another operation can never stand for one of this operation’s. A *variation* is one factor (a_j, v_j) , an argument and a direction. Every operation reports

$$\text{max_degree}, \quad \text{partial_action}(x, [(a_1, v_1), \dots, (a_k, v_k)]) = D_{a_1} \cdots D_{a_k} F(x)[v_1, \dots, v_k],$$

the mixed partial contracted against its directions, on the operation’s own domain. The default **max_degree** is 0 and the default partial stops the program: an operation that declares no calculus has none. No derivative tensor is ever stored.

Everything that is a derivative is then derived (Figure 4):

- the *linearization* $D_a S(x)[v]$ at a frozen input tuple, behind the operation interface so a minimizer sees a linear operation; exact through **partial_action** when **max_degree** ≥ 1 , by differences of two residuals otherwise—one dispatch, the primal written once;
- *Newton* as a minimizer whose Jacobian action is **tangent_of**(action), owning no derivative mathematics;
- the *chain rule to any order*: for $S(x_1(s), \dots, x_m(s))$, d^n/ds^n is assembled from the exact partial actions over integer partitions $d = [d_1, \dots, d_k]$ of n with the multinomial count $c(d) = n! / (\prod_i d_i! \prod_j \text{mult}_j!)$, partitions generated rather than tabulated; an unoccupied path derivative reads as zero;
- the *scheme’s* partials from the action’s (Section 4.9), the *stencil’s* tangent as itself (a linear map), and the *implicit function theorem* (Section 4.10).

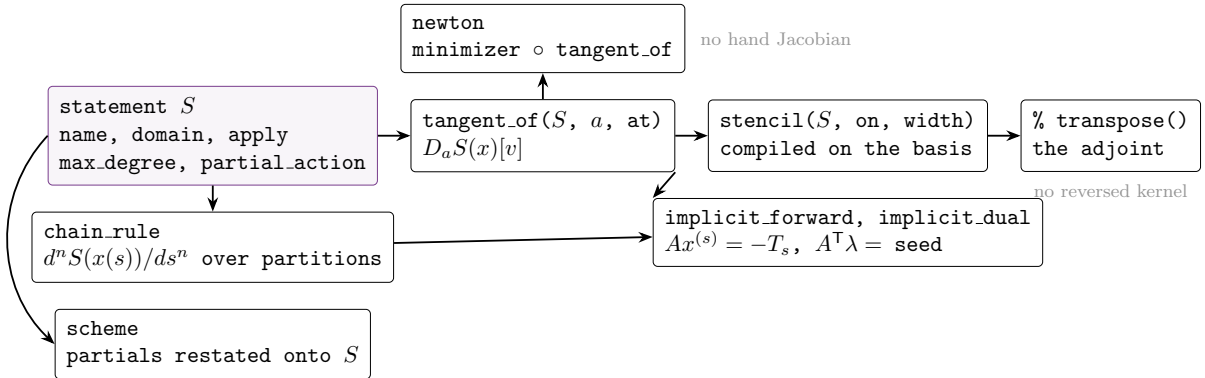


Figure 4: One primal, every derivative derived. The statement declares its exact partials once; tangent, Newton, compiled matrix, adjoint, total derivatives of any order and the implicit-function solves are built from them.

4.8 The affine sparse map

Definition 2. An affine sparse map between finite sets C (columns) and R (rows) is $\phi = (A, k) : \mathbb{R}^C \rightarrow \mathbb{R}^R$, $\phi(q) = Aq + k$, with A a weighted relation $A \subseteq R \times C$ and $k \in \mathbb{R}^R$ the constant the boundary values leave behind.

The **stencil** is this object: triples (r, c, w) , constants, two extents, and the two domains when its maker knows them. Two laws are written once:

Law 2 (composition). $(A_2, k_2) \circ (A_1, k_1) = (A_2 A_1, A_2 k_1 + k_2)$, by sparse triple product with duplicate pairs combined.

Law 3 (transpose). $(A, k)^\top = (A^\top, 0)$: the adjoint acts on the linear part.

Proposition 1. $(\phi_n \circ \dots \circ \phi_1)^\top = \phi_1^\top \circ \dots \circ \phi_n^\top$ on the linear parts; hence no reversed kernel need exist—the transpose is taken once, after the chain is composed.

The dependency graph—one edge per entry, column to row—is what a colouring and a Galerkin coarsening read; it is derived on request and not stored, the same law the directed graph applies to its own relations after a benchmark caught eager materialization costing every construction $2.2\times$. A stencil is also compiled from any operation by evaluation on the standard basis (zero state $\rightarrow$ constant; e_j minus constant $\rightarrow$ column j), with a rectangular column side when the operation’s argument lives on another domain.

The parity chain. On a directed graph (V, E) three elementary steps move values between the two sets (Figure 5): the average $S: \mathbb{R}^V \rightarrow \mathbb{R}^E$, the difference $G: \mathbb{R}^V \rightarrow \mathbb{R}^E$, $z_e = c(q_j - q_i)/h_e$, and the incidence $D: \mathbb{R}^E \rightarrow \mathbb{R}^V$, $y_v = (\text{out} - \text{in})/m_v$. An edge with no head is a boundary face; its missing end reads the boundary value into the constant. The operator of order n is

$$\text{vertex}(2k) = (DG)^k, \quad \text{vertex}(2k+1) = (DG)^k DS, \quad \text{edge}(n) = G \text{vertex}(n-1),$$

with the coefficient carried by the innermost step, so that a per-edge coefficient makes order 2 the operator $\nabla \cdot (k \nabla q)$. Each step consults an edge’s two ends, so order n reaches exactly n rings of neighbours. Exactness is claimed on the uniform chain, where the discrete formulas coincide with calculus (the second derivative is exact on a parabola, order four on x^4), and verified there.

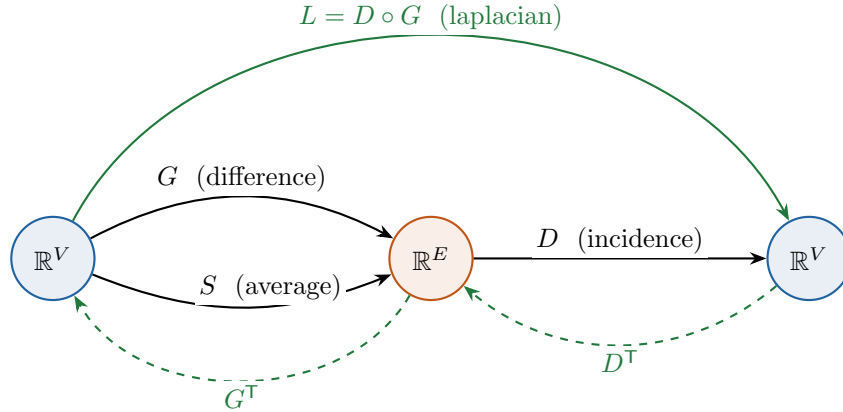


Figure 5: The parity chain. Solid arrows are the three step stencils, the green arc their composition; dashed arrows are the transposes, never written as kernels. $L^\top = (D \circ G)^\top = G^\top \circ D^\top$ is checked numerically in the demonstration `demo/law.f90`.

4.9 The scheme and the chain graph

Time is a chain graph: instants are vertices, steps are edges, a step size is a number per edge. The θ -scheme residual over an action S (which returns minus the velocity) is one operation,

$$R(q_n, \xi, q_{n-1}, q_{n-2}) = a_0 q_n + a_1 q_{n-1} + a_2 q_{n-2} + h[\theta S(q_n, \xi) + (1 - \theta) S(q_{n-1}, \xi)],$$

with $\theta = 1$ backward Euler and BDF2, $\theta = 0$ forward Euler, $\theta = \frac{1}{2}$ Crank–Nicolson, and variable-step BDF2 coefficients $a_0 = (2h_0 + h_1)/(h_0 + h_1)$, $a_1 = -(h_0 + h_1)/h_1$, $a_2 = h_0^2/(h_1(h_0 + h_1))$. The scheme’s argument space is the action’s followed by its history; every partial of R is derived

from the action's partial actions by the formula above, each variation restated on the action's argument first, so the tangent of the step equation is exact when S 's is. No scheme-specific derivative code exists, and `bdf_variable` inherits it.

4.10 The implicit function theorem, once

Let $R(x_1, \dots, x_m) = 0$ be held at its input tuple and solved in argument a , with $A = D_a R$ compiled at the tuple. Then

$$\text{forward:} \quad A x_a^{(s)} = -T_s, \quad T_s = \text{the chain-rule total of order } s \text{ with } x_a^{(s)} = 0, \quad (1)$$

$$\text{dual:} \quad A^\top \lambda = \text{seed}, \quad g_b = (D_b R)^\top \lambda \text{ for every other argument } b. \quad (2)$$

`implicit_forward` and `implicit_dual` are these two solves; the block is compiled once to a stencil, the solver is the caller's minimizer (dense direct for a 3×3 time step, CG or GMRES for a large spatial residual), and with a diagonal block nothing is compiled (an explicit scheme). The dual of any block, square or rectangular, is the transpose of the compiled tangent on that argument's own domain.

4.11 The marcher as a traversal

The marcher owns the traversal and the seeds (Figure 6). Forward: at each edge, one local solve of the scheme with the history held (by the residual at zero over $-a_0$ when $\theta = 0$, by the held minimizer otherwise). Directional: at each edge, one call of (1) with the history paths holding every solved derivative of the earlier instants. Adjoint: the converse traversal; every instant carries a seed σ_v , and at edge e with $v = e + 1$ one call of (2) answers λ_e and the duals g_k of the history blocks, after which $\sigma_{v-k} \leftarrow \sigma_{v-k} - g_k$ for k up to the reach. Adaptive marching separates mechanism from decision: the marcher computes a trial and an error estimate; a policy proposes, judges and retries from scalars alone.

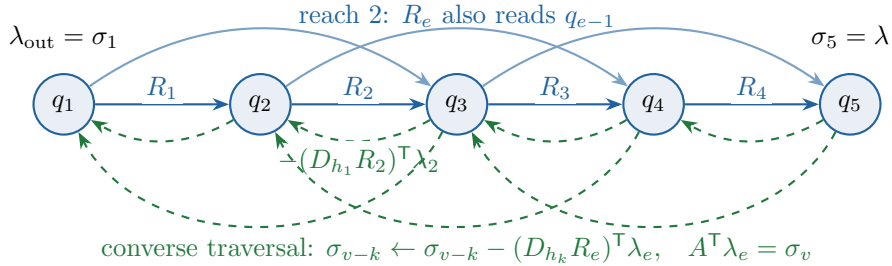


Figure 6: The marcher. Forward (blue): one implicit-function solve per edge. Converse (green, dashed): the same block transposed; each history dual is subtracted from the seed of the instant it reads. The accumulation is over the dependencies that read an instant, the form that survives on a graph that is not a chain.

4.12 Minimizers and multigrid

Every solver is a *minimizer*: attach a statement, drive its residual on the residual domain to zero by varying values on the unknown domain. The solver vocabulary—`matvec`, `inner_product`, `norm`, `sweep_order`, `diagonal`, `constant`—is defined once as delegations to the engine (the operation, the reductions, the colouring walk), so a concrete solver states only its iteration: Jacobi, Gauss–Seidel swept by colour (SOR is $\omega \neq 1$, a parameter, not a solver), conjugate gradients, restarted GMRES, dense direct elimination, Newton, and two-grid multigrid. The diagonal is probed by colour: the operator applied to one colour's indicator answers the diagonal at every member of that colour, because no two neighbours share a colour. Multigrid's coarse

operator is the fine stencil read through the aggregates—*RAP* with summing restriction and injected prolongation—obtained by relabelling the triples and combining duplicates; the suite holds the commutation square, *coarsen-then-solve returns e*.

4.13 The mesh, derived

A mesh file supplies member sets (vertices, cells, tagged boundary faces), one relation $C2V \subseteq \text{cells} \times \text{vertices}$, one field (coordinates), and labels. Everything a finite-volume solver reads is derived (Figure 7): interior faces as shared-vertex intersections of cell pairs; $V2C = C2V^\top$ by the padded transpose; $F2C = \{(f, c) : F2V(f) \subseteq C2V(c)\}$ with at most two cells per face, one cell meaning a boundary face; and the measurements as fields—centres, volumes, areas, normals, centre-to-centre deltas, interpolation weights. The mesh is the tower’s one inheritance crossing: it *is* a directed graph whose vertices are cells and whose edges are two-cell faces, boundary faces being edges without heads. No string names a measurement; an operator receives the numbers at construction and never reads the mesh. The loader reads gmsh MSH 4.1.

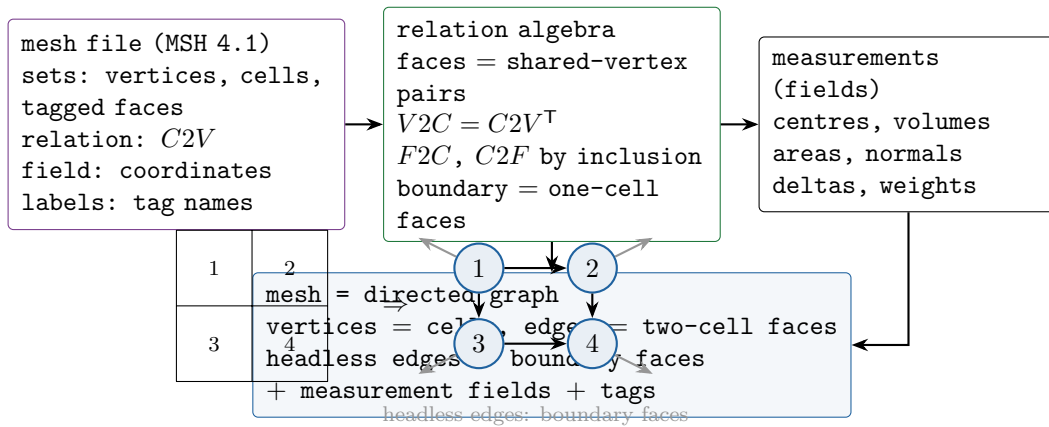


Figure 7: The mesh is derived, not stored: three primitive inputs, relation algebra, measurements as fields, and one directed graph. Cells are vertices, two-cell faces are edges, and a boundary face is an edge without a head; no ghost cell is invented.

4.14 Physics as coefficients

Physics enters as numbers and stops. A conduction law holds a tensor K and supplies $k_{\text{eff},e} = n^\top K n$ and $k_{\text{eff},e} \text{area}_e$ per face; an advection law holds a velocity and supplies v_n and $v_n \text{area}$, upwinding by the sign of the coefficient when the average step is one-sided; a Robin condition $a\phi + b\partial_n\phi = c$ on the faces of one tag eliminates the face value one-sidedly into a coefficient on the cell and a constant (Dirichlet is $a = 1, b = 0$; Neumann $a = 0, b = 1$). The string naming a tag enters exactly once. The diffusion statement translates material and walls into scales per face and values per wall, and the fitted balance compiles one value per edge—fitted exactly on the edge’s neighbourhood along the edge normal, so that skewness is used rather than assumed—into one stencil. None of these modules owns an operator, a loop over neighbourhoods, or a solve.

4.15 Reversible change

Every reversible mutation of a graph or its attached values follows one lifecycle, $\text{apply} \rightarrow \text{check} \rightarrow \text{keep} \mid \text{revert}$, under a controller that owns only the lifecycle; the change object owns the mutation, its memory and its undoing. A failure reported by apply or check is reverted and returned, not refused; a change that returns from a step without marking its work stops the program. The value map records, per graph identity, whether a value is attached, trusted, and what it is.

5 What the system can do

Load and derive a mesh. `mesh_from_gmsh(file)`: MSH 4.1 in, a directed graph with measurements and tags out; faces, incidences and geometry derived by relation algebra (Section 4.13).

State a steady problem. `diffusion_stencil(mesh, conduction(K), walls)` with Dirichlet, Neumann or Robin walls compiles material and walls into one affine operator; advection by a velocity with upwind or central averaging through the calculus’s one-sided flag.

Compose differential operators on any graph. `gradient`, `interpolation`, `divergence`, `laplacian`, and `vertex_derivative` or `edge_derivative` of any order with per-entity coefficients, spacings, measures and boundary values; `stencil_of` compiles the chain on either landing; `adjoint=.true.` applies the transpose.

Solve. Jacobi, Gauss–Seidel/SOR by colouring, CG, restarted GMRES, dense direct, Newton (exact or differenced Jacobian by the statement’s own declaration), two-grid multigrid with a Galerkin coarse operator; every solver attaches any operation and exposes `matvec`, `constant`, `diagonal`, `norm`.

Integrate in time. Forward Euler, backward Euler, BDF2 (uniform or variable step), adaptive stepping under a policy; the state may be any field, several components wide, on any domain.

Differentiate. Exact tangents of any statement in any argument; Newton Jacobians; discrete adjoints of the march with per-instant seeds and parameter duals; forward directional derivatives of any order in state or parameters; total derivatives of compositions to degree 20 by integer partitions; the implicit-function forward and dual solves on any statement, time-stepped or steady.

Cut, assemble, coarsen, refine. Partition a graph and its data into owned/borrowed parts under one relation; assemble back exactly; coarsen by aggregates and refine by parents with the one-sided law.

Reduce and broadcast. Sums, integrals, averages, norms with measures in four steps that are correct on a graph in pieces; complex and logical functionals.

Walk. Colouring, breadth-first visit order, components, depth; sources, sinks, reachability and topological order on a relation.

Change reversibly. Attach, update and revert values under one lifecycle with a consistent record.

6 Results

All measurements were made on the working tree of 22 August 2026 with `gfortran 15.2`, `-std=f2023 -O2 -fbounds-check`, on one core. The experiment programs are in `maths/demo/` and are reproduced by `make demo` and `make experiments`.

6.1 Verification suites

The repository’s test tree was removed on 21 August 2026 (369 files, 67 603 lines), so the suites are not in the tree. The evidence below is reproducible nonetheless: `maths/oracle.sh` (`make oracle`) restores the nine suites that exercise the components described above from revision `ddcf592`, applies the three mechanical interface changes of this period (Appendix A), builds them against the working tree’s library and runs them, writing only under `maths/oracle/`. On 22 August 2026 they pass: 159 property checks and 23 refusal cases (inputs that must stop the program with a stated message). Table 2 lists them with representative properties as the suites print them.

Suite	checks	representative properties verified
graph-binary	44	a tuple handed in twice is in the relation once; the view's image is the base's preimage; a view signs its own token: identity is not extension; the double transpose holds every tuple of the base
graph-field	7	a field's domain is the carrier by identity; entries count the domain
graph-ordinary	12	the naming law; module order is the dependency proof
graph-dense-direct	11	a zero leading pivot still solves exactly; a stencil compiled from an operation carries its matrix exactly; transposing twice returns the stencil
graph-minimization	7	the probed diagonal is the stencil diagonal; the sweep order never gives neighbours one colour; Jacobi lands on the exact answer
graph-differentiation	31	BDF2 variable-step coefficients $[7/5, -5/3, 4/15]$ at steps (2, 3); every degree 1–8 of $(q + \xi)^8$ matches the convolution oracle; <code>march_directional</code> matches exact rationals to degree 8; per-instant seeds accumulate to 13/9; $\langle Jv, \lambda \rangle = \langle v, J^T \lambda \rangle$ through the compiled transpose; the pairing law holds on a rectangular block
graph-marching	15	Euler lands on $(1 - h)^n$ exactly; backward Euler on $q_0/(1 + h)^n$; BDF2 quarters its error when the step halves; the reverse walk keeps the pairing $\langle \lambda, q \rangle$; tangent and adjoint meet on one gradient
graph-multigrid	7	the commutation square holds: coarsen-then-solve returns e ; the two-grid cycle meets the direct solve
graph-partition	25	every assembled edge is an edge of G and every edge of G is assembled; $\sum_k A_k(P_k(D)) = D$ on vertex and edge data; every cell owned exactly once; the wrong relation is refused by identity

Table 2: Verification suites restored by `maths/oracle.sh`, 159/159 passing on the working tree of 22 August 2026.

6.2 Time integration: orders of accuracy

The decay law $S(q, \xi) = \xi q$ (minus the velocity, so $\dot{q} = -\xi q$) with exact partial actions to all orders was marched on $[0, 1]$ from $q_0 = 1$ with $\xi = 1$ under the three rules, the implicit ones solved by Newton with GMRES inside. Table 3 and Figure 8 show the error at $t = 1$ and the observed order.

rule	N	error	order	rule	N	error	order	rule	N	error	order
forward Euler	10	1.92e-2	–	backward Euler	10	1.77e-2	–	BDF2	10	1.67e-3	–
	20	9.39e-3	1.03		20	9.01e-3	0.97		20	3.97e-4	2.07
	40	4.65e-3	1.02		40	4.55e-3	0.99		40	9.74e-5	2.03
	80	2.31e-3	1.01		80	2.29e-3	0.99		80	2.41e-5	2.01
	160	1.15e-3	1.00		160	1.15e-3	1.00		160	6.01e-6	2.01
	320	5.76e-4	1.00		320	5.74e-4	1.00		320	1.50e-6	2.00

Table 3: Error $|q_N - e^{-1}|$ and observed order for the three marching rules on $\dot{q} = -q$.

6.3 Derivatives: exactness of the adjoint and of directional derivatives

With $N = 16$ steps of backward Euler the discrete adjoint $\lambda_0 = \partial q_N / \partial q_0$ has the closed form $(1 + h\xi)^{-N}$; for BDF2 it is compared with a central finite difference of the march. Forward directional derivatives in the rate ξ to order 2 have closed forms $\partial_\xi q_N = -Nh(1 + h\xi)^{-N-1}$ and $\partial_\xi^2 q_N = N(N + 1)h^2(1 + h\xi)^{-N-2}$ (Table 4).

quantity	rule	computed	reference	vs closed form	vs finite difference
λ_0 (adjoint)	backward Euler	0.379085	$(1+h)^{-16}$	0.0	4.1×10^{-11}
λ_0 (adjoint)	BDF2	0.368507	finite difference	—	6.6×10^{-11}
$\partial_\xi q_N$	backward Euler	-0.35678619	closed form	5.6×10^{-17}	—
$\partial_\xi^2 q_N$	backward Euler	0.35678619	closed form	5.6×10^{-17}	—

Table 4: Discrete adjoint and second-order directional derivatives of the march. The adjoint is derived from the primal by transposition; the finite-difference discrepancies are the finite difference’s own truncation ($\varepsilon = 10^{-6}$).

The same two implicit-function routines, called once on the steady residual $q_i^2 - \xi = 0$ on three points with CG as the solver, return q', q'', q''' in ξ to 10^{-9} of $\sqrt{\xi}$ ’s derivatives, $\lambda = \text{seed}/2q$, and a parameter dual of -0.75 on the one-entry ξ domain, with the pairing $\langle \partial_q J, q' \rangle = -\langle (D_\xi R)^\top \lambda, 1 \rangle$ closing between the two roads (demo/law2.f90, 8/8).

6.4 Space: a Poisson problem through the constitution

On the unit square with structured quadrilateral meshes of N^2 cells generated by gmsh, the statement `diffusion.stencil(mesh, conduction(1), [four Dirichlet walls])` was solved by GMRES to a residual of 10^{-12} for the manufactured solution $u = \sin \pi x \sin \pi y$, $f = 2\pi^2 u$. Table 5 and Figure 8 give the volume-weighted L^2 and maximum errors against cell-centre values, the observed orders, and the wall-clock time of each stage.

N	cells	faces	L^2 error	order	max error	order	mesh [s]	assemble [s]	solve [s]
10	100	220	1.64e-2	—	3.11e-2	—	0.002	0.037	0.001
20	400	840	3.88e-3	2.08	7.64e-3	2.03	0.010	0.550	0.005
40	1 600	3 280	9.53e-4	2.03	1.90e-3	2.01	0.096	9.077	0.042
80	6 400	12 960	2.37e-4	2.01	4.72e-4	2.00	1.321	174.285	0.449

Table 5: Poisson on the unit square: second order in both norms. Times are one core; the scaling of each stage with cell count is discussed in Section 6.5.

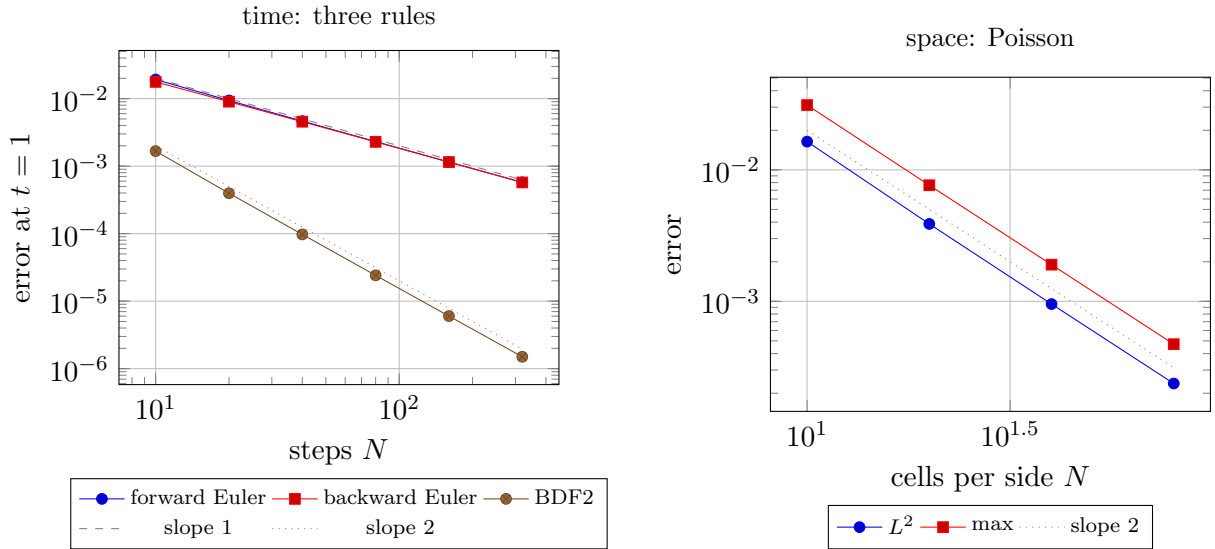


Figure 8: Convergence in time (left) and space (right).

6.5 Cost and scaling

The timings in Table 5 are the engineering result of this report. Quadrupling the cell count multiplies the three stages by (mesh derivation) 5.0, 9.6, 13.8; (operator assembly) 14.9, 16.5, 19.2; (GMRES solve) 5.0, 8.4, 10.7. Expressed as exponents of the cell count n on the last refinement, assembly scales as $n^{2.1}$, mesh derivation as $n^{1.9}$ and the unpreconditioned solve as $n^{1.7}$ (Figure 9). Assembly dominates by two orders of magnitude at 6400 cells, and its exponent is the quadratic cost of growing arrays by concatenation inside the fitted balance; a prior in-project profile attributed 97% of one fit’s 36 μ s to the stencil and CG machinery set up for a 4×4 system. The mesh derivation’s exponent points to the face intersection. None of these is a property of the mathematics; each is a representation cost to be removed (Section 10).

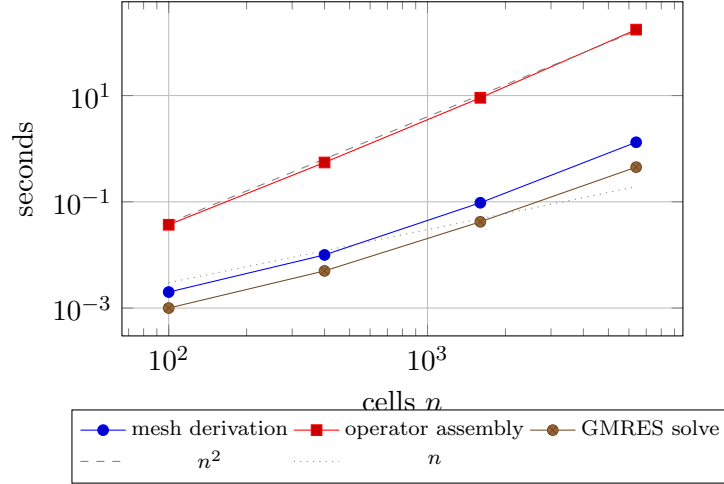


Figure 9: Wall-clock cost per stage against cell count. Assembly is quadratic; the target for every stage is linear in cells plus faces.

6.6 The source as an artefact

Table 6 records the source’s shape. Three numbers characterize the design discipline: 68% of procedures are **pure**; 191 **error stops** each state the law whose violation they refuse; and the dependency graph of the 63 modules has 245 edges (12% of the possible), a median module imported by one other, and a foundation of six modules (token, kernel graph, the set maps) whose transitive reach is 39–53 modules.

modules	63	procedures	375
source lines	21 450	pure procedures	256 (68%)
comment lines	5 858 (27%)	error stop invariants	191
abstract types	15	deferred procedures	79
extends sites	39	dependency edges (density)	245 (12%)
modules: graph, token, relation, field	1, 1, 5, 4	modules: operation, transform, map, view, util	26, 5, 7, 11, 3
commits	605	history	2018-11-15 to 2026-08-21
compiler	gfortran 15.2, -std=f2023		

Table 6: Source metrics on 22 August 2026.

6.7 The two unification slices

The measurements above were made after two changes completed on 21–22 August 2026 that made the composition and adjoint laws domain-generic: the stencil became the one affine sparse map (rectangular, with composition and transpose written once; the differential operator’s private

copy and the linearization’s basis-built dual were deleted), and the marcher’s per-instant tangent and adjoint solves became `implicit_forward` and `implicit_dual`. The suites were held at 159/159 across both; the source grew by 136 lines net, because rectangular maps with domains and the two generic solves cost more than the deleted second roads returned.

7 Discussion: what is novel, and what is different

Structure as the differentiation mechanism. The established routes to discrete adjoints differentiate programs (source transformation, overloading, compiler passes) or record them (tapes). UFVM differentiates *statements*: an operation declares its exact partial actions, and the adjoint of a compiled operator is the transpose of a composition that already exists. There is no second program to verify; the adjoint’s correctness is Proposition 1 plus the correctness of one transpose. The price is that a statement’s author must write its partials; the reward is that every derived object—Jacobian, adjoint, directional derivatives of any order—costs nothing further.

Higher-order derivatives by partitions. Taped systems obtain second derivatives by taping the tangent, and higher orders become unwieldy. The chain rule over integer partitions assembles any order directly from partial actions and is the same code for degree 1 and degree 8 (the suite checks degree 8 against a Taylor-convolution oracle in exact rationals).

Time and space as one. The implicit function theorem at a vertex is the local law of both a time step and a steady residual; the marcher is a traversal that calls it. This is the structural reading of the graph-time-integration idea—compose once, dualize instead of reimplementing, solve locally—and Sections 4.10–4.11 are its realization in code rather than a claim.

The derived mesh and the relational matrix. Deriving incidence from one relation is known; carrying identity on relations and sets, so that a partition is one relation read two ways and a wrong relation is refused by identity rather than by size, is the unusual part. Likewise a matrix as a weighted relation with a derived pattern is what makes the Galerkin coarse operator a relabelling and the adjoint a swap of two index arrays.

Parallel correctness before parallelism. The four-step reduction, the image-aware token, the owned/borrowed partition and the exact round-trip law are all designed for a graph in pieces; none of the source is parallel today. This is a deliberate ordering: the laws that make a distributed run correct are in place before the runtime exists.

What is different, honestly. The frameworks of Section 3 offer vastly more physics, scale and tooling. UFVM offers a smaller thing: a demonstration, with measurements, that a finite-volume solver with exact sensitivities of any order can be generated from eight types and a handful of laws, in 21 000 lines of a language with no runtime dependencies beyond a Fortran compiler.

8 Engineering assessment

Strengths.

- Invariants are in the type system: private components and constructor-only introduction make the kernel’s iff and token unforgeability unbreakable from outside; 191 stated refusals catch the rest at run time.
- Derivatives are verified to machine precision against closed forms, not against themselves.
- The build order is a dependency proof, checked mechanically along with the naming law on every commit.

- Every module states its laws, its costs parametrically, and what it does *not* do.

Limitations found in this assessment.

- Operator assembly is quadratic in cells (Table 5); at 6400 cells it is $400\times$ the solve.
- The Krylov solvers are unpreconditioned; multigrid is two-grid and attaches only to a compiled stencil.
- The marcher solves every implicit block with dense direct elimination by basis evaluation— n applications of the tangent per step—which is correct and exact for small states and unusable for large ones; the generic solves accept any minimizer, so the marcher needs only to pass one.
- The in-repository test tree and documentation were deleted on 21 August 2026; the 159 checks reported here run from a copy restored by `maths/oracle.sh`, not from the tree, and the pre-commit hook references a coding standard that no longer exists in the tree.
- The distributed runtime is designed but absent: no coarray statement exists in the source; the parallel build script references a deleted module.
- The mesh loader reads MSH 4.1 only; geometric refinement has no positions and asserts only what connectivity supports.
- gfortran enforces F2018 C1594, so procedures that copy a domain graph are impure; one banner claimed otherwise and was corrected.

9 Conclusions

UFVM demonstrates that a finite-volume simulation library can be generated from a small set of prime mathematical objects and that doing so turns laws into deletions: no reversed kernels, no hand Jacobians, no second CSR, no sparse-matrix class, no stored dependency pattern, no integrator-specific adjoint, no order-specific derivative code. The derived mesh, the weighted-relation matrix, the parity chain, the chain rule over partitions and the implicit function theorem at a vertex are each realized once and verified: nominal orders of accuracy in time and space, adjoints and higher-order directional derivatives exact to machine precision, and 159 structural properties holding on the current tree. The assessment is equally clear about cost: assembly is quadratic, solves are unpreconditioned, and the test tree must return to the repository. None of these touches the mathematics; all of them are representation work with a known target.

10 Future work, from an engineering perspective

1. **Linear-cost assembly.** Replace array concatenation in the fitted balance with counted allocation, and solve the per-edge 4×4 fit by a dense local kernel rather than through the stencil/CG machinery. Target: assembly linear in faces; expected gain $\geq 100\times$ at 6400 cells.
2. **Linear-cost mesh derivation.** Replace the pairwise face intersection with a hash of sorted vertex tuples; target $n^{1.0}$.
3. **Preconditioning and algebraic multigrid.** Jacobi/Gauss–Seidel preconditioners for CG/GMRES from the existing diagonal probe and colouring; multilevel aggregation from the two-grid Galerkin road.
4. **Sparse block solves in the marcher.** Pass a Krylov minimizer to `implicit_forward` and `implicit_dual` and drop the dense basis compile for large states; compile the block once per edge and reuse it across orders and the adjoint.
5. **The traversal on a DAG.** Generalize the marcher from a chain to any dependency graph in topological order, with the local statement taking one argument per in-edge; this makes space-marching and branching trajectories instances of the same traversal.

6. **More schemes by composition.** DIRK stages and Adams correctors as scaled backward steps against an externally assembled base, which the scheme already admits; Crank–Nicolson is present through $\theta = \frac{1}{2}$.
7. **The distributed runtime.** Coarray halo exchange over the partition relation, with the four-step reductions and image-aware tokens already in place; restore and measure the parallel build.
8. **Restore the test tree and documentation in-repository,** applying the three mechanical API changes of this period (derived pattern, array weights, compiled dual; Appendix A), and regenerate the architecture map from the source.
9. **Enforce the stated-only laws** ($P^{\text{TT}} = P$ extensionally, $L^{**} = L$ on the linear part, the round trip) as suite properties rather than prose.
10. **Hot-path measurement discipline.** Keep the small- n gate (the ratio of a generic path to its direct equivalent) as a test, so representation changes are admitted by measurement.

A Interface changes applied by the oracle script

The suites at `ddcf592` predate the unification of the affine map. `maths/oracle.sh` applies three mechanical changes before building them; a restored in-tree suite must apply the same.

before	after
<code>x % pattern</code>	<code>call x % dependencies(p)</code> — the pattern is derived, not stored
<code>call x % weights %</code>	<code>w = x % weights</code> (constants likewise) — plain arrays
<code>real_vector(w)</code>	
<code>call dual_by_basis(t, g, lambda,</code>	<code>stencil(t, g, width, column_domain=c, column_entries=n)</code>
<code>d)</code>	<code>then % transpose(), or implicit_dual</code>

A stencil that declares a column domain now refuses, by identity, an input field on any other set, however long; a test that builds a basis vector on a freshly derived pattern’s vertex set must build it on `x % column_domain` instead.

References

- [1] H. G. Weller, G. Tabor, H. Jasak, C. Fureby. A tensorial approach to computational continuum mechanics using object-oriented techniques. *Computers in Physics* 12(6), 1998.
- [2] T. D. Economon, F. Palacios, S. R. Copeland, T. W. Lukaczyk, J. J. Alonso. SU2: An open-source suite for multiphysics simulation and design. *AIAA Journal* 54(3), 2016.
- [3] M. Sagebaum, T. Albring, N. R. Gauger. High-performance derivative computations using CoDiPack. *ACM TOMS* 45(4), 2019.
- [4] A. Logg, K.-A. Mardal, G. N. Wells (eds.). *Automated Solution of Differential Equations by the Finite Element Method*. Springer, 2012.
- [5] F. Rathgeber et al. Firedrake: automating the finite element method by composing abstractions. *ACM TOMS* 43(3), 2016.
- [6] R. Anderson et al. MFEM: A modular finite element methods library. *Computers & Mathematics with Applications* 81, 2021.
- [7] P. E. Farrell, D. A. Ham, S. W. Funke, M. E. Rognes. Automated derivation of the adjoint of high-level transient finite element programs. *SIAM J. Sci. Comput.* 35(4), 2013.
- [8] S. K. Mitusch, S. W. Funke, J. S. Dokken. dolfin-adjoint 2018.1: automated adjoints for FEniCS and Firedrake. *J. Open Source Software* 4(38), 2019.
- [9] A. C. Hindmarsh et al. SUNDIALS: Suite of nonlinear and differential/algebraic equation solvers. *ACM TOMS* 31(3), 2005.

- [10] H. Zhang, E. M. Constantinescu, B. F. Smith. PETSc TSAjoint: a discrete adjoint ODE solver for first-order and second-order sensitivity analysis. *SIAM J. Sci. Comput.* 44(1), 2022.
- [11] L. Hascoët, V. Pascual. The Tapenade automatic differentiation tool: principles, model, and specification. *ACM TOMS* 39(3), 2013.
- [12] A. Griewank, D. Juedes, J. Utke. ADOL-C: a package for the automatic differentiation of algorithms written in C/C++. *ACM TOMS* 22(2), 1996.
- [13] W. S. Moses, V. Churavy. Instead of rewriting foreign code for machine learning, automatically synthesize fast gradients. *NeurIPS*, 2020.
- [14] J. Bradbury et al. JAX: composable transformations of Python+NumPy programs, 2018.
- [15] A. Griewank, A. Walther. *Evaluating Derivatives: Principles and Techniques of Algorithmic Differentiation*, 2nd ed. SIAM, 2008.
- [16] M. Desbrun, A. N. Hirani, M. Leok, J. E. Marsden. Discrete exterior calculus. arXiv:math/0508341, 2005.
- [17] A. N. Hirani. *Discrete Exterior Calculus*. PhD thesis, Caltech, 2003.
- [18] N. Bell, A. N. Hirani. PyDEC: software and algorithms for discretization of exterior calculus. *ACM TOMS* 39(1), 2012.
- [19] R. V. Garimella. Mesh data structure selection for mesh generation and FEA applications. *Int. J. Numer. Meth. Engng* 55, 2002.